



DOCUMENT DE CONCEPTION TECHNIQUE

Application Web de Gestion de Cité Universitaire

Contexte : Cité Universitaire — Cameroun

Stack : Next.js · Tailwind CSS · Node.js · PostgreSQL · Prisma

Cible : PWA multi-rôles · offline-first · multilingue (FR / EN / DE)

Paiement : Mobile Money (Orange Money / MTN MoMo) · rappels SMS

Version 2.0 — Juin 2026

Révision technique de la v1.0 · Réservé aux développeurs Backend & Frontend · Confidentiel

Table des matières

0.	Revue critique & évolution v1.0 → v2.0	3
1.	Introduction & objectifs	5
2.	Architecture technique	6
3.	Modèle de données (PostgreSQL / Prisma)	8
4.	Rôles & authentification	12
5.	Modules fonctionnels	14
	5.1 Facturation & coefficients · 5.2 Paiements & Mobile Money · 5.3 Notifications	
	5.4 Suggestions · 5.5 Événements · 5.6 Anniversaires · 5.7 Portfolio	
	5.8 Signal de détresse · 5.9 Posts · 5.10 Projet commun · 5.11 Prédiction	
	5.12 Maintenance/tickets · 5.13 Sondages · 5.14 Annuaire · 5.15 Documents	
6.	Dashboards Admin & Bailleur	21
7.	PWA, offline-first & internationalisation	22
8.	API REST — Endpoints	23
9.	Sécurité & bonnes pratiques	25
10.	Structure du projet (monorepo)	26
11.	Plan de déploiement	27
12.	Roadmap par phases & estimations	28

0. Revue critique & évolution v1.0 → v2.0

Cette section recense les anomalies relevées dans la conception v1.0 et les décisions retenues en v2.0. Elle sert de journal de décisions techniques pour l'équipe.

0.1 Corrections (bugs de conception)

Réf.	Problème v1.0	Décision v2.0
FIX-1	<code>montant_unit</code> déclaré <code>COMPUTED</code> : impossible en PostgreSQL, une colonne générée ne peut dépendre d'une agrégation (Σ coefficients) portant sur d'autres lignes.	Calcul porté dans le service <code>CalculFacture</code> ; persistance dans <code>facture_locataire.montant_du</code> . Aucune colonne générée inter-lignes.
FIX-2	<code>nb_personnes</code> saisi à la main et redondant avec Σ coefficients. La v1.0 ne précise jamais <i>quels</i> locataires sont rattachés à une facture.	Table de liaison <code>facture_locataire</code> = source de vérité. <code>nb_personnes</code> devient dérivé (count) et non saisi. Sélection explicite des locataires concernés (ou « tous les actifs »).
FIX-3	Table <code>paiements</code> absente du modèle alors que <code>POST /api/paiements</code> et le workflow l'invoquent. Pas de mode, ni référence, ni paiement partiel.	Table <code>paiements</code> dédiée (montant, mode, référence MoMo/OM, encaisseur, justificatif). Paiements partiels supportés ; statut recalculé.
FIX-4	<code>DECIMAL(12, 2)</code> pour des montants en XAF : le franc CFA n'a pas de subdivision en usage.	Montants en entiers : <code>BIGINT</code> (XAF). Les coefficients restent <code>DECIMAL(4, 2)</code> .
FIX-5	Contradiction « eau / électricité » dans la prédiction : la formule <code>I * P + TVA + LC + T</code> est propre à l'électricité.	Module de prédiction paramétré par type de charge (électricité, eau...), chaque type ayant sa formule. Renommé « estimation » (déterministe, pas du ML).
FIX-6	Audit trail cité en §9 mais absent du modèle de données.	Table <code>audit_logs</code> formalisée + middleware d'audit sur toutes les actions sensibles.

0.2 Risques de conception revus

RISQUE SÉCURITÉ — SIGNAL DE DÉTRESSE

La désactivation automatique du bouton après « 3 signaux en 3 jours » s'applique à une **fonction de sécurité vitale**. Bloquer un résident réellement en danger plusieurs jours d'affilée est inacceptable et engage la responsabilité de la cité. **v2.0** : on remplace le blocage dur par (a) un anti-abus progressif (throttle + cooldown), (b) une **mise en revue admin** au lieu d'une coupure, (c) la possibilité pour l'admin de lever toute restriction instantanément. Le ban manuel admin reste possible mais journalisé.

MOT DE PASSE ≥ 4 CARACTÈRES

Avec un login = simple `username`, 4 caractères exposent à du bruteforce trivial. **v2.0** : minimum **8 caractères** (la 1^{re} connexion impose le changement), rate-limit strict, verrouillage temporaire, et **2FA TOTP optionnel** pour l'Admin. Le « 4 caractères » initial n'est toléré que pour le mot de passe *temporaire* généré, jamais conservé.

RECONDUCTION AUTOMATIQUE DES FACTURES

Reconduire en silence peut propager un montant erroné indéfiniment. **v2.0** : la reconduction crée un brouillon `is_reconducted=true` **non publié**, notifie l'admin pour validation, et un compteur `reconduction_streak` déclenche une alerte au-delà de 2 mois consécutifs.

0.3 Adaptations au contexte camerounais (ajouts v2.0)

- + Mobile Money** Enregistrement natif des paiements **Orange Money / MTN MoMo** (mode + référence + capture). Intégration API en option (phase 2).
- + SMS** Canal **SMS** (passerelle locale type Nexah/Twilio) pour les rappels d'échéance et signaux de détresse — plus fiable que l'email localement.
- + Offline-first** Stratégie PWA renforcée pour connexion intermittente et data coûteuse : cache agressif, compression d'images, file de synchronisation différée.
- + Compteurs** Modélisation des **chambres / blocs / compteurs** pour répartir eau & électricité par compteur physique.
- ? Allemand** Les langues officielles sont FR/EN. Le DE est conservé (architecture i18n prête) mais marqué **optionnel** : à confirmer selon le public réel, vu le coût de traduction/maintenance.

0.4 Nouveaux modules fonctionnels proposés

5.12 Maintenance / tickets (pannes)

5.13 Sondages & votes

5.14 Annuaire des résidents (recherche par compétence)

5.15 Documents & règlement intérieur

Reçus de paiement PDF

Suivi de cotisation au projet commun

Export comptable CSV/Excel

Graphiques de consommation par locataire

Mode sombre

1. Introduction & objectifs

1.1 Contexte

CampusGest est une **Progressive Web App (PWA)** de gestion complète d'une cité universitaire au Cameroun. Elle digitalise et centralise les interactions entre l'**Administration**, le **Bailleur** et les **Locataires** : facturation et paiements, communication, vie communautaire et suivi financier. L'objectif d'usage est une **ouverture quotidienne** par chaque résident, soutenue par les notifications push et SMS.

1.2 Objectifs principaux

#	Objectif	Priorité
O1	Facturation automatisée avec coefficients individuels & répartition par compteur	CRITIQUE
O2	Paielements & suivi des encaissements (Mobile Money, reçus PDF)	CRITIQUE
O3	Communication interne multicanal (push, SMS, email, annonces, suggestions)	HAUTE
O4	Suivi financier temps réel pour Admin & Bailleur	HAUTE
O5	Sécurité communautaire (signal de détresse encadré, alertes)	HAUTE
O6	Vie communautaire (événements, anniversaires, portfolio, posts, projets)	MOYENNE
O7	Estimation des charges du mois suivant	MOYENNE
O8	PWA offline-first multilingue (FR / EN / DE)	HAUTE

1.3 Périmètre & contraintes

- Seul l'**administrateur** est créé au déploiement (seed idempotent). Tout autre compte est créé via l'interface Admin.
- Le mot de passe temporaire généré doit être changé dès la première connexion ; le mot de passe permanent fait ≥ 8 caractères.
- Les coefficients sont saisis mensuellement ; à défaut, `n = 1` pour tous.
- En l'absence de nouvelle facture pour un type donné, un **brouillon reconduit** du mois précédent est proposé à l'admin (jamais publié automatiquement).
- Les abus de signal de détresse passent en **revue admin** (jamais de coupure silencieuse de la sécurité).
- Montants en **XAF entiers**. Devise et formats respectent la locale de chaque utilisateur.

2. Architecture technique

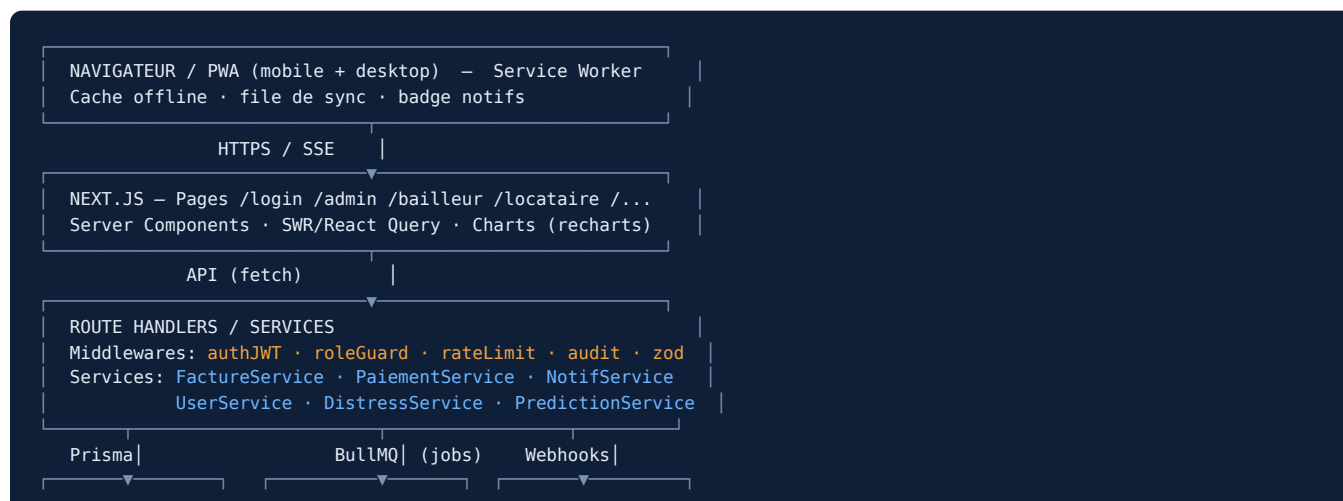
2.1 Vue d'ensemble de la stack

Couche	Technologie	Rôle
Frontend	<code>Next.js 14</code> (App Router)	SSR/SSG, Server Components, routes API
Styling / UI	<code>Tailwind CSS 3</code> + <code>shadcn/ui</code>	Design responsive, thème (#1A3C6E / #E8821E), mode sombre
Backend	<code>Node.js</code> + Next.js Route Handlers	Logique métier, endpoints REST (couche Services)
Base de données	<code>PostgreSQL 15</code>	Stockage relationnel principal
ORM	<code>Prisma</code>	Modélisation, migrations, requêtes typées
Validation	<code>Zod</code>	Validation/sanitisation de toutes les entrées API (schemas partagés)
Auth	<code>JWT</code> (access 15 min) + refresh (HttpOnly) + <code>bcrypt</code>	Sessions, 2FA TOTP admin optionnel
Temps réel	<code>SSE</code> (défaut) / <code>Socket.io</code> (option)	Notifications live ; SSE privilégié (compatible reverse-proxy)
Push / SMS / Email	Web Push (VAPID) · SMS (Nexah/Twilio) · Nodemailer/SMTP	Multicanal, selon préférences & criticité
Filaire d'attente	<code>BullMQ</code> + <code>Redis</code>	Jobs (alertes, SMS, reconductions, anniversaires), retries
PWA	<code>next-pwa</code> (Workbox)	Service Worker, offline-first, install A2HS
i18n	<code>next-intl</code>	FR / EN / DE, détection auto, persistée en base
Stockage fichiers	Cloudinary (ou MinIO self-host)	Images portfolio, posts, projets, justificatifs
Observabilité	Sentry + logs <code>pino</code> + health checks	Erreurs, traces, métriques
Déploiement	VPS Ubuntu + Docker Compose / Railway	Nginx reverse-proxy + SSL Let's Encrypt
CI/CD	GitHub Actions	Lint, typecheck, tests, build, déploiement

CHOIX TEMPS RÉEL

SSE est retenu par défaut : unidirectionnel (serveur→client), suffisant pour des notifications, simple à passer derrière Nginx, et plus économe qu'une connexion WebSocket persistante sur un VPS modeste. Socket.io reste une option si un besoin bidirectionnel (chat, présence) émerge.

2.2 Architecture logicielle — couches



PostgreSQL 15	Redis + Workers	Services tiers
users·factures	CRON · alertes	Cloudinary·SMTP
paiements·...	SMS · push	SMS·MoMo·Push

2.3 Portails par rôle

Rôle	Route de base	Accès principal
Admin	/admin/...	Dashboard global, gestion users, factures & paiements, tous modules, modération
Bailleur	/bailleur/...	Dashboard financier, factures par locataire, encaissements, historique, annonces, export
Locataire	/locataire/...	Mes factures & paiements, notifications, profil/portfolio, communauté

3. Modèle de données (PostgreSQL / Prisma)

Conventions : PK `UUID` (`gen_random_uuid()`), `created_at` / `updated_at` en `TIMESTAMPTZ` , montants en **XAF entiers** (`BIGINT`), suppression = désactivation (soft delete).

3.1 users

Colonne	Type	Description
id	UUID PK	Identifiant
username	VARCHAR(50) UNIQUE	Login
password_hash	VARCHAR(255)	Hash bcrypt (rounds 12)
role	ENUM	<code>admin</code> <code>bailleur</code> <code>locataire</code>
full_name	VARCHAR(100)	Nom complet
email / phone	VARCHAR	Email (UNIQUE, optionnel) / téléphone (clé SMS)
room_id	UUID FK chambres	Chambre occupée (nullable) — NEW
language	VARCHAR(5) = 'fr'	fr / en / de
first_login	BOOLEAN = true	Force changement credentials
is_active	BOOLEAN = true	Compte actif
totp_secret	TEXT NULL	2FA TOTP (admin) — NEW
distress_disabled	BOOLEAN = false	Détresse bloquée par admin (manuel)
distress_review	BOOLEAN = false	En revue anti-abus (n'empêche pas l'urgence) — NEW
birthday	DATE NULL	Anniversaire (opt-in)
birthday_public	BOOLEAN = false	Consentement diffusion — NEW
notif_prefs	JSONB	Canaux préférés {push, sms, email} par type — NEW
push_subscription	JSONB NULL	Abonnement Web Push (endpoint+clés, > simple token)
created_at / updated_at	TIMESTAMPTZ	Audit temps

3.2 chambres & compteurs **NEW**

chambres

Colonne	Type / note
id	UUID PK
bloc	VARCHAR(20) — bâtiment/aile
numero	VARCHAR(20)
compteur_elec_id	UUID FK compteurs NULL
capacite	INT
is_active	BOOLEAN

compteurs

Colonne	Type / note
id	UUID PK
type	ENUM eau electricite
libelle	VARCHAR(60)
dernier_index	BIGINT — relevé courant
scope	ENUM cite bloc chambre

3.3 factures

Colonne	Type	Description
id	UUID PK	Identifiant
type	VARCHAR(60)	Eau, Électricité, Nettoyage, Contribution...
montant_total	BIGINT	Montant brut total (XAF entier) — FIX-4
somme_coeff	DECIMAL(8,2)	Σ des coefficients des locataires rattachés (dérivé)
base_unitaire	BIGINT	<code>round(montant_total / somme_coeff)</code> — calculé au service — FIX-1
compteur_id	UUID FK compteurs NULL	Compteur rattaché (eau/élec) — NEW
mois	VARCHAR(7)	<code>YYYY-MM</code>
date_limite	DATE	Échéance de paiement
statut_pub	ENUM	<code>brouillon</code> <code>publiee</code> — la reconduction reste brouillon — NEW
is_reconducted	BOOLEAN = false	Reconduit du mois précédent
reconduction_streak	INT = 0	Nb de mois reconduits d'affilée (garde-fou) — NEW
created_by	UUID FK users	Admin créateur
created_at	TIMESTAMP TZ	

FIX-1 / FIX-2

`somme_coeff` et `base_unitaire` sont écrits par `FactureService` lors du calcul/recalcul, à partir des lignes `facture_locataire` rattachées — et non par une colonne `GENERATED` (techniquement impossible ici). `nb_personnes` de la v1.0 est supprimé : c'est `count(facture_locataire)`.

3.4 facture_locataire (répartition individuelle)

Colonne	Type	Description
id	UUID PK	
facture_id	UUID FK factures	Facture concernée
locataire_id	UUID FK users	Locataire concerné (<i>UNIQUE avec facture_id</i>)
coefficient	DECIMAL(4,2) = 1.0	Coefficient individuel (n)
montant_du	BIGINT	<code>round(base_unitaire * coefficient)</code>
montant_paye	BIGINT = 0	Cumul payé (paiements partiels) — NEW
statut	ENUM	<code>en_attente</code> <code>partiel</code> <code>paye</code> <code>retard</code>
date_paiement	DATE NULL	Date du solde complet

AJUSTEMENT D'ARRONDI

La somme des `round()` peut différer de `montant_total` de quelques XAF. Le service applique l'écart résiduel sur la dernière ligne pour garantir $\Sigma \text{montant_du} = \text{montant_total}$ (réconciliation exacte).

3.5 paiements NEW

Colonne	Type	Description
id	UUID PK	
facture_locataire_id	UUID FK	Ligne réglée
montant	BIGINT	Montant encaissé (partiel possible)
mode	ENUM	<code>especes</code> <code>orange_money</code> <code>mtn_momo</code> <code>virement</code>
reference	VARCHAR(80) NULL	Réf. transaction MoMo/OM
justificatif_url	TEXT NULL	Capture/reçu (Cloudinary)
recorded_by	UUID FK users	Encaisseur (Admin)
recu_pdf_url	TEXT NULL	Reçu généré (PDF)
created_at	TIMESTAMP TZ	

3.6 notifications

Colonne	Type	Description
id	UUID PK	
sender_id	UUID FK users NULL	NULL si système
target_role	VARCHAR(20) NULL	<code>all</code> <code>admin</code> <code>bailleur</code> <code>locataire</code>
target_user	UUID FK users NULL	Destinataire individuel
type	VARCHAR(40)	<code>annonce</code> <code>alerte_facture</code> <code>anniversaire</code> <code>detresse</code> <code>lecture</code> <code>evenement</code> <code>prediction</code> <code>maintenance</code> <code>sondage</code>
title / body	VARCHAR(200) / TEXT	Contenu (clés i18n + variables)
channels	JSONB	Canaux émis {inapp, push, sms, email} — NEW
is_read	BOOLEAN = false	
created_at	TIMESTAMP TZ	

3.7 Tables communautaires & système

Table	Colonnes clés
suggestions	author_id, contenu, ordre, is_read_admin, bailleur_visible(def true), read_at, created_at
evenements	creator_id, titre, description, date_event, heure, statut(en_attente approuve rejete), created_at
portfolios	user_id, bio, competences(JSONB), diplomes(JSONB), realisations(JSONB), contact, email_pro, dispo_recommandation(bool)
distress_signals	sender_id, sent_at, latitude, longitude, geo_consent(bool), resolved(bool), resolved_by
posts_info	author_id, titre(≤100), description(≤300), image_url, is_hidden, created_at
projets_communs	created_by, titre, description, objectifs(JSONB), vision, besoins_financiers(BIGINT), montant_contribution(BIGINT), image_url, visible_roles(JSONB)
projet_contributions NEW	projet_id, user_id, montant(BIGINT), paiement_id, created_at — suivi de cagnotte
predictions_facture	mois, type, indice_diff, prix_unit, tva, loc_compteur, transport, montant_calcule(BIGINT), montant_reel(BIGINT NULL), created_at

Table	Colonnes clés
maintenance_tickets NEW	author_id, room_id, categorie(plomberie electricite autre), description, image_url, statut(ouvert en_cours resolu), priorite, assigned_to, created_at
sondages / sondage_votes NEW	question, options(JSONB), is_open, created_by · (sondage_id, user_id, choix) UNIQUE
documents NEW	titre, fichier_url, categorie(reglement pv autre), visible_roles(JSONB), uploaded_by
audit_logs FIX-6	user_id, action, resource, resource_id, ip_address, user_agent, metadata(JSONB), created_at

4. Rôles & authentification

4.1 Flux d'authentification

1. L'Admin se connecte avec les credentials seedés (mot de passe initial à usage unique, changement forcé).
2. L'Admin crée les comptes Bailleur/Locataire ; un mot de passe temporaire est généré et transmis hors-bande (SMS/remise).
3. À la 1^{re} connexion → redirection vers « Changer mes identifiants » (nouveau mot de passe ≥ 8 caractères).
4. Hash `bcrypt` (salt rounds = 12).
5. Login réussi → **access token** JWT HS256 (15 min) + **refresh token** (7 j, cookie HttpOnly/SameSite=Strict).
6. Payload access : `{ sub, role, ver, iat, exp }` (`ver` = compteur d'invalidation pour révocation).
7. Middleware centralisé : vérification signature/exp, puis `roleGuard` par endpoint.
8. 2FA TOTP optionnel pour l'Admin (étape supplémentaire post-mot de passe).

DURCISSEMENT VS V1.0

Access court (15 min) + refresh rotatif > le JWT 24h de la v1.0 : limite la fenêtre de compromission. Révocation via `users.token_version`.

4.2 Matrice des permissions

Fonctionnalité	Admin	Bailleur	Locataire
Créer / modifier / désactiver utilisateurs	Oui	Non	Non
Saisir factures & coefficients	Oui	Non	Non
Enregistrer un paiement	Oui	Non	Non
Voir détail factures (tous)	Oui	Oui	Non
Voir ses propres factures / payer	Oui	Oui	Oui
Export comptable	Oui	Oui	Non
Envoyer notifications / annonces globales	Oui	Oui	Non
Voir toutes les suggestions	Oui	Cond.	Non
Envoyer une suggestion	Oui	Oui	Oui
Approuver / rejeter événement	Oui	Non	Non
Créer événement / post / sondage-vote	Oui	Oui	Oui
Gérer signal de détresse (revue/ban)	Oui	Non	Non
Envoyer signal de détresse	Oui	Oui	Oui
Publier / modérer projet commun, documents	Oui	Non	Non
Ouvrir / suivre ticket maintenance	Oui	Oui	Oui
Saisir prédiction de charge	Oui	Non	Non
Dashboard	Global	Financier	Perso

Cond. = visible uniquement si l'Admin a activé la visibilité Bailleur pour la suggestion. Cette mécanique n'est jamais exposée aux auteurs.

5. Modules fonctionnels

5.1 Facturation & coefficients

Cœur financier. L'Admin crée des factures mensuelles par type, rattache les locataires concernés, attribue des coefficients et fixe l'échéance.

Répartition :

```
base_unitaire = montant_total / Σ(coefficients des locataires rattachés)
montant_du(i) = round(base_unitaire × coefficient(i)) (écart d'arrondi reporté sur la dernière ligne)
```

Exemple — Eau = 60 000 XAF, 4 locataires (coeff 1, 1, 0.5, 1.5) → $\Sigma = 4.0$ → base = 15 000

A(1)=15 000 · B(1)=15 000 · C(0.5)=7 500 · D(1.5)=22 500 → **Total = 60 000 XAF ✓**

Règles métier

- Aucun coefficient saisi ce mois → tous à `n = 1`.
- Aucune facture saisie pour un type → **brouillon reconduit** du mois précédent (non publié), notifié à l'Admin pour validation (cf. §0.2).
- Alerte (push + SMS + email selon préférences) au locataire en défaut, au Bailleur et à l'Admin **J-2** avant l'échéance, puis **J0** si non réglé, puis rappels J+3 / J+7.
- Bailleur : vue complète (par locataire, total par type, historique, export). Locataire : ses propres factures uniquement.
- Gestion du départ en cours de mois : un locataire désactivé est exclu du calcul des factures **non publiées** ; les factures publiées restent figées.

Workflow backend

Étape	Action
POST /api/factures	Crée la facture (type, montant_total, mois, date_limite, [compteur], [locataires])
POST /api/factures/:id/coefficients	Saisit/maj les coefficients par locataire
SERVICE CalculFacture	Calcule somme_coeff, base_unitaire, montant_du ; réconcilie l'arrondi
POST /api/factures/:id/publish	Publie (sort du brouillon) et déclenche les notifications
JOB échéances (quotidien)	Calcule retards, envoie alertes J-2 / J0 / J+n
POST /api/paiements	Enregistre un paiement (total/partiel), recalcule le statut, génère le reçu PDF

5.2 Paiements & Mobile Money NEW

- Enregistrement Admin du paiement : montant, **mode** (espèces, Orange Money, MTN MoMo, virement), **référence transaction**, justificatif (capture).
- Paiements partiels** : `montant_paye` cumulé ; statut `partiel` tant que `montant_paye < montant_du`.
- Reçu PDF** généré et stocké, téléchargeable par le locataire.
- Phase 2 (option)** : intégration API Mobile Money (collecte/confirmation par webhook) pour réduire la saisie manuelle.

5.3 Notifications & annonces

Admin et Bailleur émettent annonces/notifications reçues par tous. Les notifications système sont générées automatiquement. Chaque utilisateur définit ses **canaux préférés** (`notif_prefs`) ; les alertes critiques (détresse, échéance) forcent push + SMS.

Type	Émetteur	Destinataires	Déclencheur
ANNONCE	Admin / Bailleur	Tous	Message général
ALERTE_FACTURE	Système (job)	Locataire + Admin + Bailleur	Échéance imminente / retard
ANNIVERSAIRE	Système (job)	Tous (actifs)	Rappel J-7 (si opt-in)
DETRESSE	Utilisateur	Tous (actifs)	Signal d'urgence
LECTURE	Système	Auteur suggestion	Admin a ouvert la suggestion

Type	Émetteur	Destinataires	Déclencheur
EVENEMENT	Système	Tous	Approbation / rejet
PREDICTION	Système	Tous	Publication d'une estimation
MAINTENANCE	Système	Auteur + Admin	Changement de statut d'un ticket

5.4 Boîte à suggestions

- Tout utilisateur soumet des suggestions numérotées (champ `ordre`, réordonnables par glisser-déposer).
- Vues par l'Admin ; l'Admin active/désactive la visibilité Bailleur par suggestion. **Non exposé aux auteurs** : ils savent seulement que « l'administration » verra leur suggestion.
- Notification de **lecture** envoyée à l'auteur dès l'ouverture par l'Admin (`read_at` horodaté).
- Si l'Admin coupe la visibilité Bailleur d'une suggestion, celui-ci cesse de la voir immédiatement.

5.5 Programmation d'événements

Tout utilisateur propose un événement (Titre, Jour, Heure, Description). L'Admin **approuve** ou **rejette** ; le statut (badge coloré EN_ATTENTE / APPROUVÉ / REJETÉ) est visible par tous. Notification au créateur dès décision.

5.6 Anniversaires

- Renseignement opt-in (`birthday` + consentement `birthday_public`).
- Job quotidien : détection à **J-7** → notification collective (prénom + jour, **sans l'année**).

5.7 Portfolio professionnel

Page optionnelle visible par tous, pour valoriser compétences/réalisations et encourager les recommandations.

Section	Contenu
Biographie	Texte court
Diplômes	Établissement, année, mention (JSONB)
Compétences	Tags libres (JSONB)
Réalisations	Images + titre + description
Contact	Téléphone, email pro, réseaux
Disponibilité	Ouvert aux recommandations : oui/non

5.8 Signal de détresse REVU §0.2

- Déclenchement : **5 clics rapides** sur le bouton « Détresse » (anti-accident) + confirmation optionnelle de géoloc (consentement explicite).
- Diffusion push + SMS à **tous les utilisateurs actifs** en temps réel ; historisé dans `distress_signals`.
- **Anti-abus (sans couper la sécurité)** : au-delà d'un seuil (ex. 3 signaux / 3 jours), le compte passe en `distress_review` → l'Admin est alerté pour arbitrage ; **le signal continue de partir**, mais marqué « à vérifier ».
- L'Admin peut bannir/réactiver manuellement (`distress_disabled`) — action journalisée (audit).
- Suivi de résolution : un signal peut être marqué `resolved` par l'Admin.

5.9 Tableau d'informations (posts images)

- Image obligatoire (JPEG/PNG, ≤ 5 Mo, Cloudinary, compression auto). Titre ≤ 100, description ≤ 300 caractères.
- Fil commun trié par date décroissante ; l'Admin peut masquer tout post inapproprié (`is_hidden`).

5.10 Projet commun & cotisations cotisation

L'Admin crée/publie un projet visible par les rôles sélectionnés. Champs : Titre, image de profil, Description, Objectifs (liste), Vision, Besoins financiers, Montant de contribution/personne, Rôles destinataires (multi-select).

- **Suivi de cagnotte** : table `projet_contributions` + barre de progression (collecté / objectif), liée aux paiements.

5.11 Prédiction / estimation de charge FIX-5

L'Admin saisit les paramètres ; le système calcule l'estimation du mois suivant, **par type de charge**, publiée à tous pour anticipation.

Électricité : $\text{Montant} = (I \times P) + \text{TVA} + \text{LC} + T$

I = différence d'indices (kWh = index_fin - index_début) · P = prix du kWh · TVA · LC = location compteur · T = transport

Eau : $\text{Montant} = (I_{\text{eau}} \times P_{\text{eau}}) + \text{frais_fixes}$ (formule propre, paramétrable par type)

Stocké dans `predictions_facture` avec `montant_reel` renseigné a posteriori pour **comparer estimé vs réel** et affiner.

5.12 Maintenance & tickets NEW

Distinct du signal de détresse : signalement non urgent (plomberie, électricité, mobilier...). Catégorie, description, photo, statut (ouvert → en cours → résolu), priorité, assignation. Notifications à chaque changement de statut. Utile au quotidien d'une cité.

5.13 Sondages & votes NEW

Décisions communautaires rapides (ex. date d'un événement, choix d'un prestataire). Question + options, un vote par utilisateur, résultats en temps réel, clôture par l'Admin.

5.14 Annuaire des résidents NEW

Vue dérivée des portfolios : recherche par compétence/diplôme, filtre « disponible aux recommandations ». Renforce le réseautage (objectif communautaire).

5.15 Documents & règlement intérieur NEW

Espace de documents partagés (règlement, PV de réunion, formulaires), visibilité par rôle, versionné, téléchargeable hors-ligne (cache PWA).

6. Dashboards Admin & Bailleur

6.1 Dashboard Administrateur

Locataires actifs

Factures du mois

Taux de paiement %

Impayés en retard

Tickets maintenance ouverts

- **Graphique financier** : barres mensuelles facturé vs encaissé (12 mois) + répartition par mode de paiement.
- **Tableau impayés** : locataire, montant dû, jours de retard, dernier rappel envoyé.
- **Activité récente** : 5 dernières actions admin (issu de `audit_logs`).
- **Alertes actives** : signaux de détresse (dont « à vérifier »), anniversaires à venir, événements en attente, tickets prioritaires.
- **Raccourcis** : Créer facture · Créer utilisateur · Publier annonce · Saisir prédiction · Enregistrer paiement.

6.2 Dashboard Bailleur

Total facturé (mois)

Total encaissé

Taux de recouvrement

Nombre d'impayés

- **Tableau mensuel** par locataire : Eau / Électricité / Autres · statut · date paiement · mode.
- **Historique** : 12 mois paginés, filtres par type/statut/mode + **export CSV/Excel/PDF**.
- **Alertes** : locataires en retard (J-2, J0, J+n).
- **Tendance** : courbe d'évolution facturé/encaissé sur 6 mois.

7. PWA, offline-first & internationalisation

7.1 Progressive Web App

- `next-pwa` (Workbox) ; `manifest.json` : nom, icônes 192/512, thème `#1A3C6E` , `display: standalone` .
- **Offline-first** : précache du shell ; *StaleWhileRevalidate* pour les listes, *NetworkFirst* pour les données fraîches, *CacheFirst* pour images/documents.
- **File de synchronisation** (Background Sync / IndexedDB) : les actions hors-ligne (paiement saisi, suggestion, ticket) sont mises en file et rejouées au retour réseau.
- **Économie de data** : images servies en WebP/AVIF responsives, lazy-loading, pagination serveur partout.
- Prompt d'installation A2HS après 2 visites ; badge de notifications non lues (Badging API).
- Push via Web Push + VAPID, abonnement en `push_subscription` .

LIMITE IOS

Le Web Push iOS ne fonctionne **que sur iOS 16.4+ et uniquement si la PWA est installée** (A2HS). Fallback SMS prévu pour les alertes critiques sur iPhone non installé.

7.2 Internationalisation (FR / EN / DE)

- `next-intl` , fichiers `/messages/{fr,en,de}.json` ; détection auto à la 1^{re} visite, surchargeable et persistée (`users.language`).
- Tout est traduit : UI, erreurs, notifications, SMS, emails (selon langue du destinataire).
- Formats date/nombre/monnaie (XAF) par locale.
- **DE optionnel** (cf. §0.3) : structure prête, activation selon le public réel.

8. API REST — Endpoints

Toutes les entrées validées par Zod. Réponses paginées (`?page&limit`) sur les listes. Audit sur actions sensibles.

8.1 Auth & utilisateurs

Méth.	Route	Rôle	Description
POST	<code>/api/auth/login</code>	Public	Connexion → access + refresh
POST	<code>/api/auth/refresh</code>	Cookie	Rotation du token
POST	<code>/api/auth/logout</code>	Authentifié	Invalide le refresh
POST	<code>/api/auth/2fa/setup verify</code>	Admin	Activation TOTP
POST	<code>/api/auth/change-credentials</code>	Authentifié	Change username/password
GET / POST	<code>/api/users</code>	Admin	Liste / crée
PUT / DELETE	<code>/api/users/:id</code>	Admin	Modifie / désactive (soft)
GET / PUT	<code>/api/users/:id/profile</code>	Auth / Proprio	Profil & préférences notif

8.2 Factures & paiements

Méth.	Route	Rôle	Description
GET	<code>/api/factures</code>	Admin/Bailleur	Liste (filtres mois/type/statut)
POST	<code>/api/factures</code>	Admin	Crée (brouillon)
PUT	<code>/api/factures/:id</code>	Admin	Modifie
POST	<code>/api/factures/:id/coefficients</code>	Admin	Coefficients par locataire
POST	<code>/api/factures/:id/publish</code>	Admin	Publie + notifie
GET	<code>/api/factures/locataire/:id</code>	Admin/Bailleur/Proprio	Factures d'un locataire
GET	<code>/api/factures/historique</code>	Admin/Bailleur	Historique + export
POST	<code>/api/paiements</code>	Admin	Enregistre (mode, réf, partiel) + reçu PDF
GET	<code>/api/paiements/:id/recu</code>	Proprio/Admin	Télécharge le reçu PDF

8.3 Notifications, suggestions & communauté

Méth.	Route	Rôle	Description
GET	<code>/api/notifications</code> · <code>/stream</code> (SSE)	Authentifié	Mes notifs / flux live
POST	<code>/api/notifications</code>	Admin/Bailleur	Annonce
PATCH	<code>/api/notifications/:id/read</code>	Authentifié	Marque lue
GET / POST	<code>/api/suggestions</code>	Admin(Bailleur cond.) / Auth	Liste / crée
PATCH	<code>/api/suggestions/:id/visibility</code>	Admin	Visibilité Bailleur

Méth.	Route	Rôle	Description
GET/POST · PATCH	/api/evenements · /:id/statut	Auth · Admin	Événements / décision
GET / PUT	/api/portfolios/:id · /api/portfolios	Authentifié	Voir / éditer
GET	/api/annuaire?skill=	Authentifié	Recherche résidents
POST · PATCH	/api/distress · /api/users/:id/distress-ban	Auth · Admin	Signal / ban
GET / POST	/api/posts	Authentifié	Fil d'infos / publier
GET / POST	/api/projets · /:id/contribuer	Selon rôle · Admin	Projets / cotisation
GET / POST	/api/predictions	Auth · Admin	Estimations
GET/POST · PATCH	/api/tickets · /:id/statut	Auth · Admin	Maintenance
GET/POST · POST	/api/sondages · /:id/vote	Auth	Sondages / vote
GET / POST	/api/documents	Selon rôle · Admin	Documents partagés

9. Sécurité & bonnes pratiques

Domaine	Mesure
Authentification	JWT HS256 access 15 min + refresh 7 j (HttpOnly, SameSite=Strict), rotation & révocation par <code>token_version</code> . 2FA TOTP admin.
Mots de passe	≥ 8 caractères + bcrypt(12). Changement forcé à la 1 ^{re} connexion. Verrouillage temporaire après échecs.
Rate limiting	<code>/auth/login</code> : 10 essais / 15 min / IP + backoff par compte. Throttle global sur l'API.
Validation	Zod sur 100% des entrées (schemas partagés front/back). Aucune donnée non validée persistée.
Injection SQL	Prisma paramétré ; <code>\$queryRaw</code> tagué uniquement pour rapports, jamais d'interpolation brute.
Autorisation	<code>roleGuard</code> + vérification d' ownership (un locataire n'accède qu'à ses ressources). Tests d'accès systématiques.
Upload	MIME + taille (≤ 5 Mo) vérifiés côté serveur, ré-encodage image, pas d'exécutable. URLs signées.
Transport	HTTPS obligatoire (Let's Encrypt), HSTS. CORS en liste blanche explicite.
En-têtes	CSP stricte (anti-XSS), <code>X-Frame-Options</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> (géoloc).
Secrets	<code>.env</code> non versionné : <code>JWT_SECRET</code> , <code>DB_URL</code> , VAPID, SMTP, SMS, Cloudinary, MoMo.
Données perso	Géoloc & anniversaire en opt-in ; minimisation ; pas de PII dans les logs.
Audit	<code>audit_logs</code> sur actions sensibles (user, action, ressource, IP, UA, timestamp).
Observabilité	Sentry (erreurs), <code>pino</code> structuré + rotation, health checks <code>/api/health</code> .
Sauvegardes	<code>pg_dump</code> quotidien chiffré → stockage objet (R2/S3), restauration testée.

10. Structure du projet (monorepo Turborepo)

```
campusgest/
├─ apps/
│  └─ web/                                # Application Next.js (App Router)
│     └─ app/
│        ├── (auth)/login/page.tsx
│        ├── admin/{dashboard,factures,paielements,users,...}/page.tsx
│        ├── bailleur/{dashboard,factures,historique}/page.tsx
│        ├── locataire/{dashboard,factures,profil}/page.tsx
│        ├── communaute/{evenements,portfolios,annuaire,projets,posts,
│        │               sondages,tickets,documents}/page.tsx
│        └─ api/{auth,factures,paielements,notifications,distress,...}/route.ts
│     ├── components/{ui,layouts,charts,forms}/    # shadcn/ui + recharts
│     ├── lib/{prisma.ts,auth.ts,rbac.ts,notifications.ts,calculations.ts,
│     │       sms.ts,push.ts,pdf.ts,i18n.ts}
│     ├── messages/{fr,en,de}.json
│     └─ public/{manifest.json,icons/,sw.js}
├─ packages/
│  ├── db/          # Prisma: schema.prisma + migrations/ + seed.ts (idempotent)
│  ├── shared/      # Types TS + schemas Zod partagés front/back
│  └─ workers/      # BullMQ: echeances, anniversaires, reconductions, sms, push
├─ scripts/cron-jobs.ts
├─ docker-compose.yml    # web + postgres + redis + nginx
├─ .env.example
└─ turbo.json / package.json
```

11. Plan de déploiement

11.1 Environnements

Env.	Description	Hébergement
DEV	Local, Docker Compose (web + postgres + redis)	localhost
STAGING	Préproduction, tests d'intégration / e2e	Railway / Render
PROD	Production, backups quotidiens chiffrés	VPS Ubuntu (Hetzner/OVH) + Docker + Nginx

11.2 Étapes (production)

1. Provisionner VPS Ubuntu 22.04 LTS : SSH durci, `ufw`, `fail2ban`, mises à jour auto.
2. Installer Docker + Compose ; cloner le repo.
3. Configurer `.env` (DB, JWT, VAPID, SMTP, SMS, Cloudinary, MoMo).
4. `docker-compose up -d` (web + postgres + redis + nginx).
5. `npx prisma migrate deploy` puis `npx prisma db seed` (admin idempotent — upsert).
6. Nginx + SSL Let's Encrypt (`certbot --nginx`), HSTS.
7. Workers BullMQ + CRON système (échéances J-2/J0/J+n, anniversaires J-7, reconductions).
8. Test du flux complet : login admin → 2FA → user → facture → publication → paiement (MoMo) → reçu → push/SMS.
9. Sauvegardes `pg_dump` chiffrées → R2/S3 ; test de restauration.
10. GitHub Actions : lint + typecheck + tests + build + déploiement sur `main`.

11.3 Variables d'environnement

```
DATABASE_URL=postgresql://user:pass@localhost:5432/campusgest
REDIS_URL=redis://localhost:6379
JWT_SECRET=          JWT_REFRESH_SECRET=
NEXT_PUBLIC_APP_URL=https://votre-domaine.cm
VAPID_PUBLIC_KEY=  VAPID_PRIVATE_KEY=  VAPID_SUBJECT=mailto:admin@votre-domaine.cm
CLOUDINARY_CLOUD_NAME=  CLOUDINARY_API_KEY=  CLOUDINARY_API_SECRET=
SMTP_HOST=smtp.gmail.com SMTP_PORT=587 SMTP_USER= SMTP_PASS=
SMS_PROVIDER=nexah SMS_API_KEY= SMS_SENDER_ID=CAMPUSGEST
MOMO_OM_API_KEY=  MOMO_MTN_API_KEY=  # phase 2 (option)
ADMIN_DEFAULT_USERNAME=admin ADMIN_DEFAULT_PASSWORD=  # temporaire, changé au 1er login
```

11.4 Notes pour l'équipe

1. TypeScript `strict: true` partout. Types & schemas Zod partagés via `packages/shared`.
2. Tests unitaires (Jest/Vitest) **obligatoires** sur `CalculFacture` (arrondis, reconduction) et `NotificationService` ; e2e (Playwright) sur le flux facturation→paiement.
3. Seed admin idempotent (upsert).
4. Pagination serveur systématique.
5. SSE par défaut ; Socket.io seulement si besoin bidirectionnel avéré.
6. Tester la PWA sur Android (Chrome) et iOS (Safari, A2HS) ; valider le fallback SMS.

12. Roadmap par phases & estimations

Phase	Périmètre	Livrables clés
P0 Socle	Auth/RBAC, users, i18n, PWA shell, CI/CD, Docker	Login, gestion users, portails, déploiement staging
P1 Financier	Factures, coefficients, calcul, paiements + MoMo (saisie), reçus PDF, alertes J-2/J0, dashboards	Cœur métier opérationnel (O1, O2, O4)
P2 Communication	Notifications multicanal (push/SMS/email), annonces, suggestions, SSE	O3
P3 Sécurité & communauté	Signal de détresse encadré, tickets maintenance, événements, anniversaires	O5, début O6
P4 Communauté +	Portfolio, annuaire, posts, projet commun + cotisation, sondages, documents	O6
P5 Estimation & finitions	Prédiction par type, estimé vs réel, exports comptables, mode sombre, durcissement	O7, audit sécurité
P6 Option	Intégration API Mobile Money (webhooks), graphiques de consommation	Automatisation paiements

DÉCISIONS À CONFIRMER AVANT P1

- **Allemand (DE)** : maintenir les 3 langues ou démarrer FR/EN et activer DE plus tard ?
- **Mobile Money** : saisie manuelle (P1) suffit-elle au lancement, ou intégration API dès le départ ?
- **Plusieurs bailleurs** possibles, ou un seul ? Impacte la matrice de permissions.
- **Chambres/compteurs** : périmètre réel (un seul compteur global vs plusieurs par bloc) ?